

Java 9 — Modules (JPMS)

Java Version: 9

Year: 2017

Feature: Modules / Java Platform Module System (JPMS)

UniqueClassName: ModulesJPMS

1. Definition

JPMS (Java Platform Module System) is a module system introduced in **Java 9** through **Project Jigsaw**.

It allows a large Java application to be divided into **modules**, where each module explicitly declares:

- which packages it **exports**
- which other modules it **requires**
- which services it **provides**
- which services it **uses**

The module declaration is normally placed in a file named:

module-info.java

Basic syntax

```
module module.name {  
    requires another.module;  
    exports package.name;  
}
```

For example:

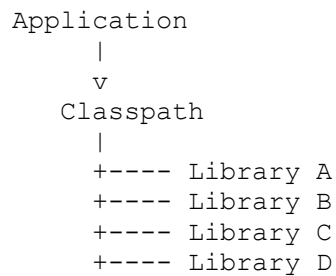
```
module com.example.app {  
    requires com.example.service;  
    exports com.example.app;  
}
```

Here:

- module com.example.app → declares the module.
 - requires → establishes a dependency on another module.
 - exports → makes a package accessible to other modules.
 - Packages that are **not exported** remain strongly encapsulated from other modules.
-

2. Why JPMS Was Introduced

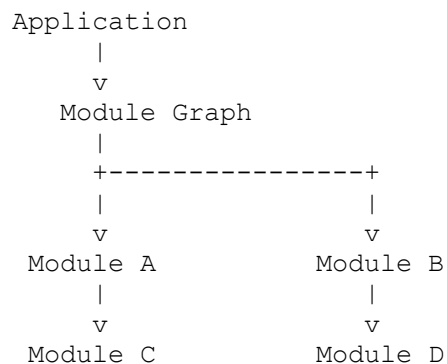
Before Java 9, applications commonly depended on the **classpath**.



The classpath had several problems:

- Dependencies were not explicitly declared.
- Classes could accidentally depend on internal implementation packages.
- Large applications became difficult to maintain.
- Duplicate classes could cause classpath conflicts.
- Encapsulation was weaker.
- The Java runtime itself was a very large monolithic system.

JPMS introduced a more structured approach:



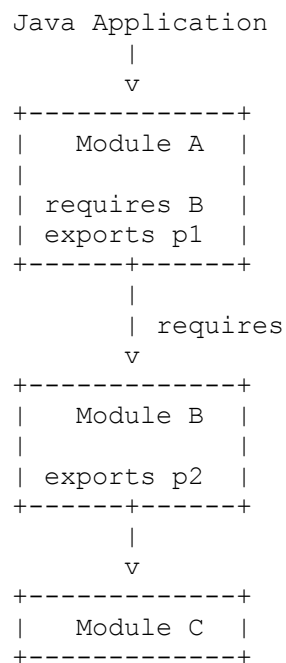
3. Important JPMS Concepts

Keyword / Concept	Purpose
module	Declares a module
requires	Declares dependency on another module
exports	Makes a package accessible to other modules
opens	Opens packages for reflective access

provides	Declares an implementation of a service
uses	Declares that a module consumes a service
requires transitive	Passes a dependency to modules that depend on the current module
requires static	Compile-time dependency that is optional at runtime
module-info.java	Module descriptor

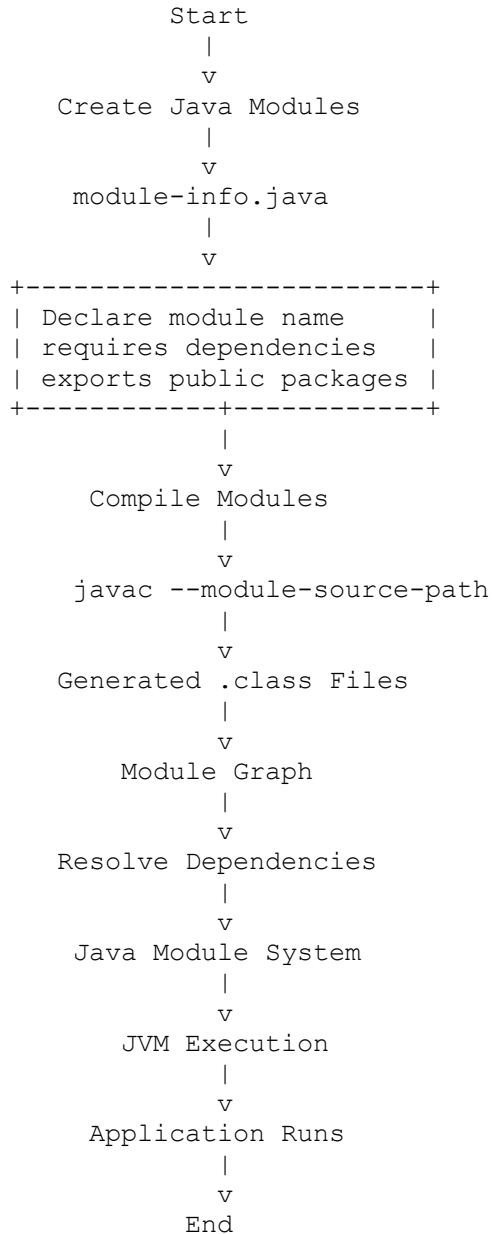
4. JPMS Architecture

A simplified architecture is:



The important difference from a traditional classpath is that the **module graph explicitly describes dependencies**.

5. Flowchart — JPMS Compilation and Execution



6. Java Code Example

Let's create two modules:

```
Module A
|
| requires
v
Module B
```

Module B provides a service class, and Module A uses it.

Project Structure

```
JPMSExample/
├── src/
│   ├── com.example.service/
│   │   ├── module-info.java
│   │   └── com/
│   │       ├── example/
│   │       │   └── service/
│   │       │       └── GreetingService.java
│   └── com.example.app/
│       ├── module-info.java
│       └── com/
│           ├── example/
│           │   └── app/
│           │       └── ModulesJPMS.java
```

Module B — Service Module

module-info.java

```
module com.example.service {
    exports com.example.service;
}
```

This means:

Module: com.example.service

Exports:
com.example.service

Other modules can access classes inside the exported package.

GreetingService.java

```
package com.example.service;

public class GreetingService {

    public String getMessage() {
        return "Hello from JPMS Module!";
    }
}
```

```
}
```

7. Module A — Application Module

module-info.java

```
module com.example.app {  
    requires com.example.service;  
}
```

The statement:

```
requires com.example.service;
```

means:

```
com.example.app  
  |  
  | depends on  
  v  
com.example.service
```

ModulesJPMS.java

```
package com.example.app;  
  
import com.example.service.GreetingService;  
  
public class ModulesJPMS {  
    public static void main(String[] args) {  
        GreetingService service = new GreetingService();  
        System.out.println(service.getMessage());  
    }  
}
```

Output

Hello from JPMS Module!

8. Compilation Flow

Assume the project has this structure:

```
src/
├── com.example.service/
│   ├── module-info.java
│   └── com/example/service/GreetingService.java
└── com.example.app/
    ├── module-info.java
    └── com/example/app/ModulesJPMS.java
```

Compile the modules:

```
javac -d out --module-source-path src -m com.example.service,com.example.app
```

The compiler understands:

```
com.example.app
  |
  | requires
  v
com.example.service
```

9. Execution

Run the application with:

```
java --module-path out -m com.example.app/com.example.app.ModulesJPMS
```

The command has two important parts:

--module-path out

tells the JVM where the compiled modules are located.

And:

-m com.example.app/com.example.app.ModulesJPMS

means:

Module:

com.example.app

Main Class:
com.example.app.ModulesJPMS

10. Complete Execution Flow

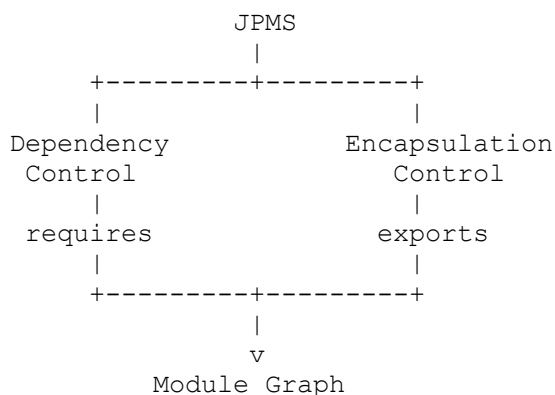
```
module-info.java
|
v
+-----+
| com.example.service |
| exports service     |
+-----+
|
| requires
^
+-----+-----+
| com.example.app   |
| ModulesJPMS.java  |
+-----+-----+
|
v
javac
|
v
Compiled Modules
|
v
Module Path
|
v
Module Resolution
|
v
JVM
|
v
ModulesJPMS.main()
|
v
GreetingService
|
v
"Hello from JPMS Module!"
```

11. JPMS vs Traditional Classpath

Feature	Classpath	JPMS
Introduced	Earlier Java versions	Java 9
Dependency declaration	Mostly implicit	Explicit
Dependency keyword	—	requires
Package exposure	Generally accessible	Controlled with exports
Strong encapsulation	Limited	Yes
Module descriptor	No	module-info.java
Module graph	No	Yes
Split large applications	Difficult	Easier
Runtime modularity	Limited	Supported
Java platform modularization	No	Yes

12. Key Idea

The central idea of JPMS is:



So, instead of simply putting everything on one large **classpath**, Java 9 allows applications to explicitly define **modules and their relationships**.

In one sentence

JPMS is Java's module system introduced in Java 9 that provides explicit dependency management, strong encapsulation, and a module graph through module-info.java.

Java 9 — module-info.java

Java Version: 9

Year: 2017

Feature: module-info.java

UniqueClassName: ModuleInfo

1. Definition

module-info.java is a special Java source file introduced in **Java 9** as part of the **Java Platform Module System (JPMS)**.

It is called the **module descriptor** because it describes the structure and requirements of a module.

It tells the Java compiler and JVM:

- the **name** of the module
- which modules it **requires**
- which packages it **exports**
- which packages it **opens** for reflection
- which services it **provides**
- which services it **uses**

Basic syntax

```
module moduleName {  
    requires anotherModule;  
    exports packageName;  
}
```

For example:

```
module com.example.app {  
    requires com.example.service;  
    exports com.example.app;  
}
```

Here:

```
module com.example.app
    |
    +-- requires com.example.service
    |
    +-- exports com.example.app
```

2. Why module-info.java Was Introduced

Before Java 9, Java applications primarily used the **classpath**.

```
Application
  |
  v
Classpath
  |
  +---- Library A
  +---- Library B
  +---- Library C
```

Dependencies were not formally described by the Java language itself.

Java 9 introduced:

```
Application
  |
  v
Module
  |
  +---- requires Module A
  |
  +---- requires Module B
  |
  +---- exports Package X
```

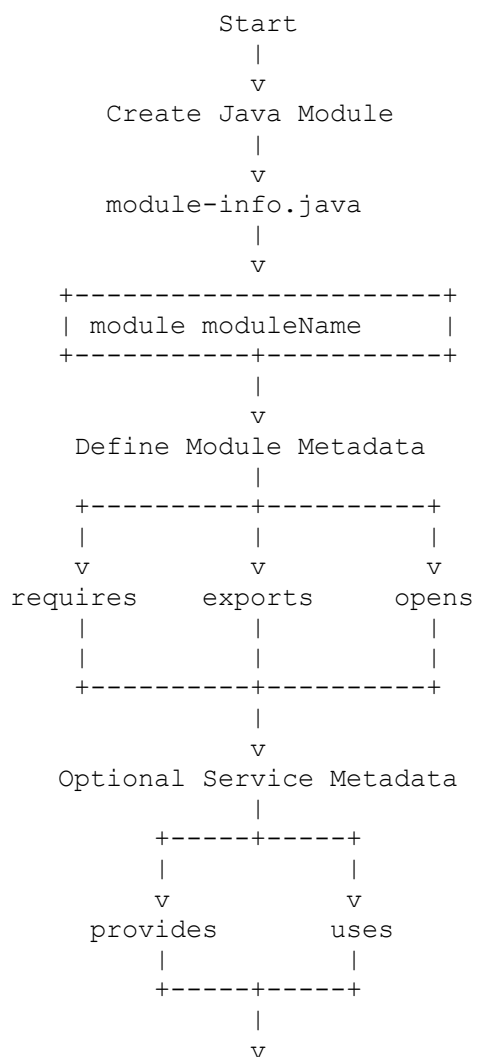
Therefore, module-info.java acts as the **declaration/contract of a module**.

3. Important Keywords in module-info.java

Keyword	Purpose
module	Declares the module and its name
requires	Specifies another module that this module depends on

exports	Makes a package accessible to other modules
opens	Allows reflective access to a package
provides	Declares a service implementation
uses	Declares that the module consumes a service
requires transitive	Makes a dependency available to modules that depend on the current module
requires static	Declares an optional compile-time dependency

4. Flowchart — module-info.java



```
javac compiler
  |
  v
Module Descriptor
  |
  v
Module System
  |
  v
JVM
  |
  v
End
```

5. Java Code Example

Let's create two modules:

```
com.example.service
  ^
  |
  requires
  |
com.example.app
```

Project Structure

```
ModuleInfoExample/
├── src/
│   ├── com.example.service/
│   │   ├── module-info.java
│   │   └── com/
│   │       ├── example/
│   │       │   └── service/
│   │       │       └── GreetingService.java
│   └── com.example.app/
│       ├── module-info.java
│       └── com/
│           ├── example/
│           │   └── app/
│           │       └── ModuleInfo.java
```

6. Service Module

com.example.service/module-info.java

```
module com.example.service {  
    exports com.example.service;  
}
```

This declares:

Module name:
com.example.service

Exported package:
com.example.service

GreetingService.java

```
package com.example.service;  
  
public class GreetingService {  
    public String getMessage() {  
        return "Hello from Service Module!";  
    }  
}
```

7. Application Module

com.example.app/module-info.java

```
module com.example.app {  
    requires com.example.service;  
}
```

The important statement is:

```
requires com.example.service;
```

It creates a dependency:

```
com.example.app  
  |  
  | requires  
  v  
com.example.service
```

Because com.example.service exports its package:

exports com.example.service;

the application module can access GreetingService.

8. Main Java Class

ModuleInfo.java

```
package com.example.app;

import com.example.service.GreetingService;

public class ModuleInfo {

    public static void main(String[] args) {

        GreetingService service = new GreetingService();

        System.out.println(service.getMessage());

    }

}
```

Output

Hello from Service Module!

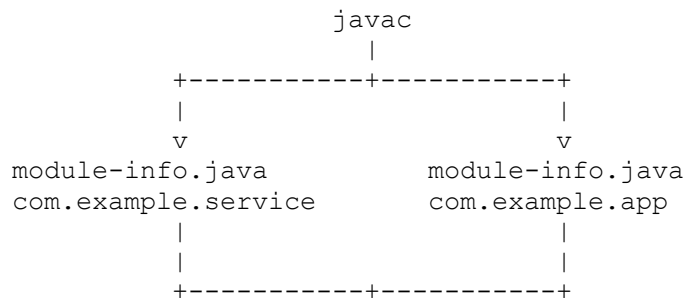
9. Compilation

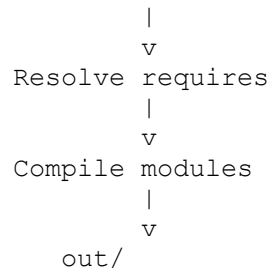
Compile both modules using:

```
javac -d out --module-source-path src -m com.example.service,com.example.app
```

The compiler reads both module-info.java files.

Conceptually:





10. Execution

Run the application:

```
java --module-path out -m com.example.app/com.example.app.ModuleInfo
```

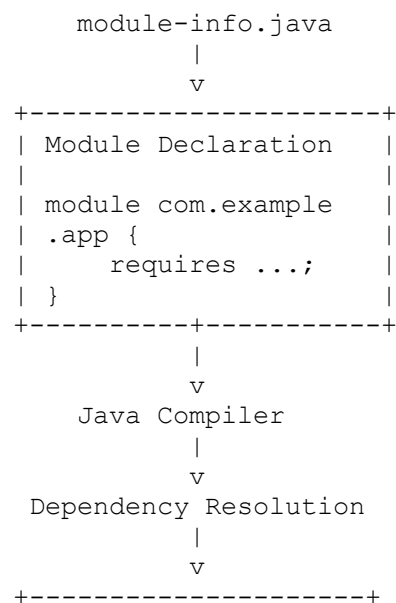
Breakdown:

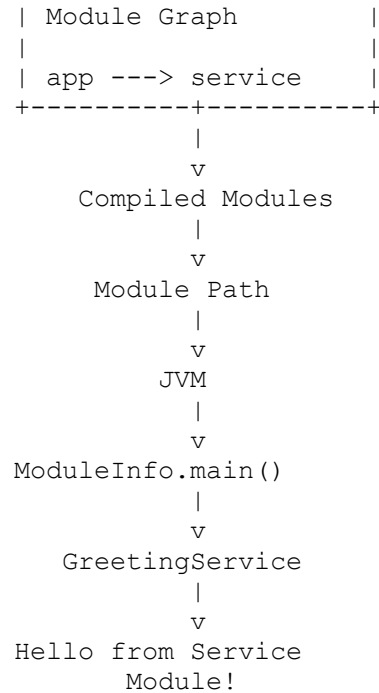
```
--module-path out
|
+-- Location of compiled modules

-m com.example.app/com.example.app.ModuleInfo
|
+-- Module: com.example.app
|
+-- Main class: com.example.app.ModuleInfo

```

11. Complete Execution Flow





12. module-info.java vs Normal .java File

Feature	module-info.java	Normal .java
Purpose	Describes a module	Defines classes/interfaces/etc.
Introduced	Java 9	Java 1.0
Contains classes	No	Yes
Contains module declaration	Yes	No
Defines dependencies	requires	import does not define module dependency
Controls package visibility	exports	No
Controls reflection	opens	No
Defines services	provides / uses	No
Compiled result	Module descriptor	.class files

13. Important Difference: requires vs import

This is particularly important.

`requires com.example.service;`

and

`import com.example.service.GreetingService;`

are **not the same thing**.

requires

`requires com.example.service;`

defines a **module-level dependency**.

Module A
|
| requires
v
Module B

import

`import com.example.service.GreetingService;`

makes a particular type available by its simple name inside the Java source code.

Java source
|
+-- import GreetingService

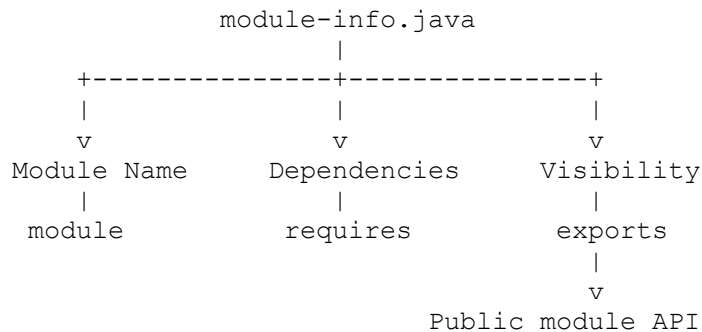
Therefore:

module-info.java
|
+---- requires → Module dependency

Java source file
|
+---- import → Type/package name usage

14. Key Concept

Think of `module-info.java` as the **identity card and dependency contract of a Java module**.



In one sentence

`module-info.java` is Java 9's module descriptor that explicitly defines a module's name, dependencies, exported/opened packages, and service relationships for the JPMS.

Java 9 — JShell

Java Version: 9

Year: 2017

Feature: JShell

UniqueClassName: JShell

1. Definition

JShell is an interactive **Read-Eval-Print Loop (REPL)** tool introduced in **Java 9**.

It allows developers to write and execute Java code **one statement at a time without creating a .java file, compiling it separately, and then running it manually**.

JShell is particularly useful for:

- Learning Java
- Testing Java syntax
- Experimenting with APIs

- Quickly testing expressions
- Debugging small pieces of logic
- Exploring classes and methods

Traditional Java execution

Write .java file



javac



.class file



java



Output

JShell execution

Enter Java code



JShell



Evaluate immediately



Result

2. What Does REPL Mean?

REPL stands for:

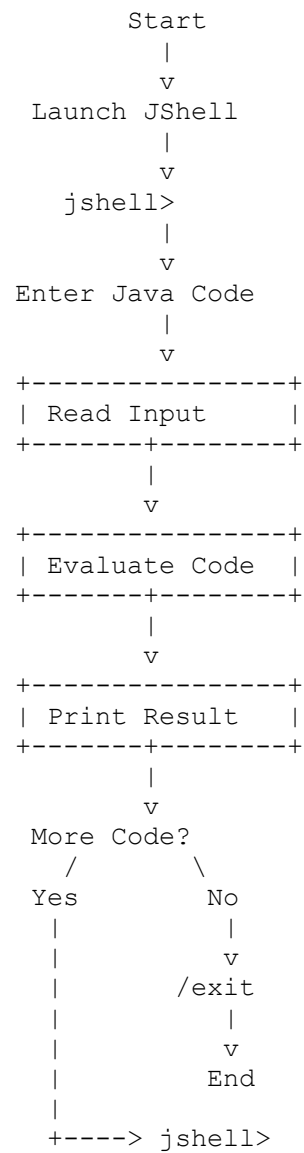
Letter	Meaning	JShell operation
R	Read	Reads your Java input
E	Eval	Evaluates the input
P	Print	Displays the result
L	Loop	Waits for the next input

Therefore:

Read → Evaluate → Print → Loop



3. JShell Flowchart



4. Starting JShell

From the command line, execute:

```
jshell
```

You will see something similar to:

```
| Welcome to JShell -- Version 9
```

```
| For an introduction type: /help intro
```

```
jshell>
```

The:

```
jshell>
```

prompt means JShell is ready to accept Java code.

5. Simple Java Code Example

Enter:

```
int a = 10;
```

JShell accepts it immediately.

Then:

```
int b = 20;
```

Then:

```
a + b
```

Output:

```
$3 ==> 30
```

The complete interaction looks like:

```
jshell> int a = 10  
a ==> 10
```

```
jshell> int b = 20  
b ==> 20
```

```
jshell> a + b  
$3 ==> 30
```

Notice that you didn't need:

```
public class Test {  
    public static void main(String[] args) {  
        // code  
    }  
}
```

```
}
```

JShell provides the interactive environment for you.

6. Java Class Example in JShell

You can also define methods and classes.

Method

```
int square(int n) {  
    return n * n;  
}
```

Then call it:

```
square(5)
```

Output:

```
$2 ==> 25
```

Class

```
class Student {  
    String name;  
  
    Student(String name) {  
        this.name = name;  
    }  
  
    void display() {  
        System.out.println("Student: " + name);  
    }  
}
```

Create an object:

```
Student s = new Student("Anurag");
```

Call the method:

```
s.display();
```

Output:

```
Student: Anurag
```

7. JShell Example Using the JShell Class Name

For a conceptual example corresponding to your UniqueClassName: JShell, we can demonstrate a small Java program that performs the same basic calculation we tested interactively:

```
public class JShell {  
    public static void main(String[] args) {  
        int a = 10;  
        int b = 20;  
  
        int sum = a + b;  
  
        System.out.println("a = " + a);  
        System.out.println("b = " + b);  
        System.out.println("Sum = " + sum);  
    }  
}
```

Output:

```
a = 10  
b = 20  
Sum = 30
```

Note: This class is only a normal Java example named JShell; it is **not** the implementation of the JDK's JShell tool.

8. JShell Commands

JShell also provides special commands beginning with `/`.

For example:

Display help

```
/help
```

Display variables

```
/vars
```

Display methods

/methods

Display types

/types

Display history

/history

Exit JShell

/exit

A typical session:

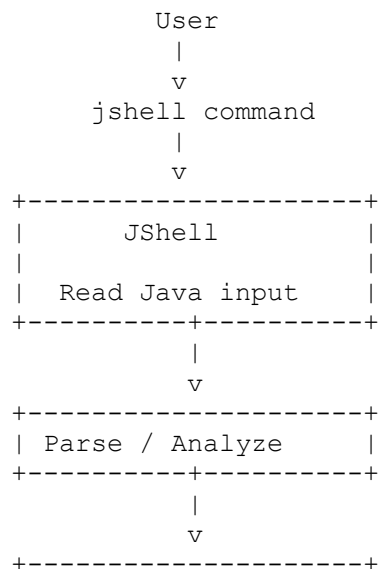
```
jshell> int x = 100  
x ==> 100
```

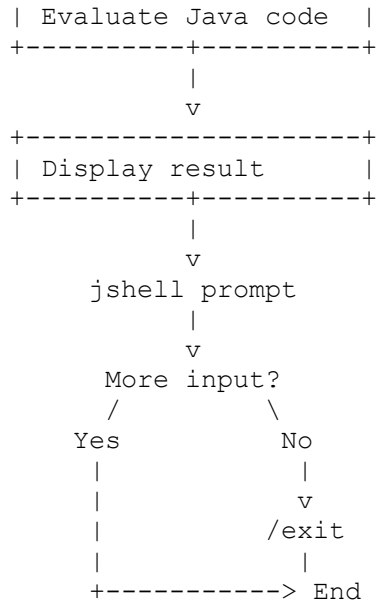
```
jshell> /vars  
|    int x = 100
```

```
jshell> /history  
int x = 100  
/vars  
/history
```

```
jshell> /exit  
|  Goodbye
```

9. JShell Execution Flow

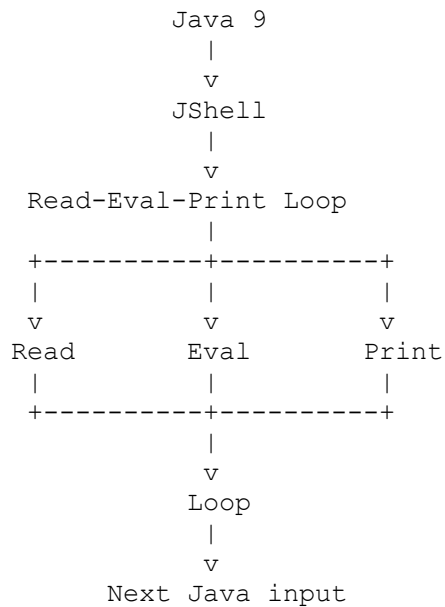




10. JShell vs Traditional Java Program

Feature	Traditional Java	JShell
Introduced	Java 1.0	Java 9
Type code interactively	No	Yes
.java file required	Normally yes	No
Explicit compilation workflow	Yes	Handled interactively
main() required	Usually	No
Quick experimentation	Less convenient	Very convenient
REPL	No	Yes
Commands such as /vars	No	Yes
Best suited for	Applications	Experimentation/learning/testing

11. Complete Conceptual Flow



Key Point

JShell is Java 9's interactive REPL tool that lets you immediately read, evaluate, and print the results of Java expressions, statements, methods, and classes without requiring the traditional edit → compile → run workflow.

Java 9 — Collection Factory Methods

Java Version: 9

Year: 2017

Feature: Collection Factory Methods

UniqueClassName: CollectionFactories

1. Definition

Collection factory methods were introduced in **Java 9** to provide a short, convenient, and readable way to create **immutable collections**.

The main factory methods are:

List.of(...)

```
Set.of(...)
Map.of(...)
Map.ofEntries(...)
```

They eliminate much of the boilerplate that was previously required to create unmodifiable collections.

Before Java 9

For example, creating an unmodifiable list commonly required:

```
List<String> names =
    Collections.unmodifiableList(
        Arrays.asList("Java", "Python", "C++")
    );
```

Java 9

The same idea becomes:

```
List<String> names = List.of("Java", "Python", "C++");
```

The factory methods return collections that **cannot be structurally modified**.

2. Main Collection Factory Methods

Factory Method	Collection Created	Example
List.of()	Immutable List	List.of("A", "B")
Set.of()	Immutable Set	Set.of("A", "B")
Map.of()	Immutable Map	Map.of("A", 1, "B", 2)
Map.ofEntries()	Immutable Map	Map.ofEntries(...)

3. Basic Syntax

List

```
List<String> list = List.of("Java", "Python", "C++");
```

Set

```
Set<String> set = Set.of("Java", "Python", "C++");
```

Map

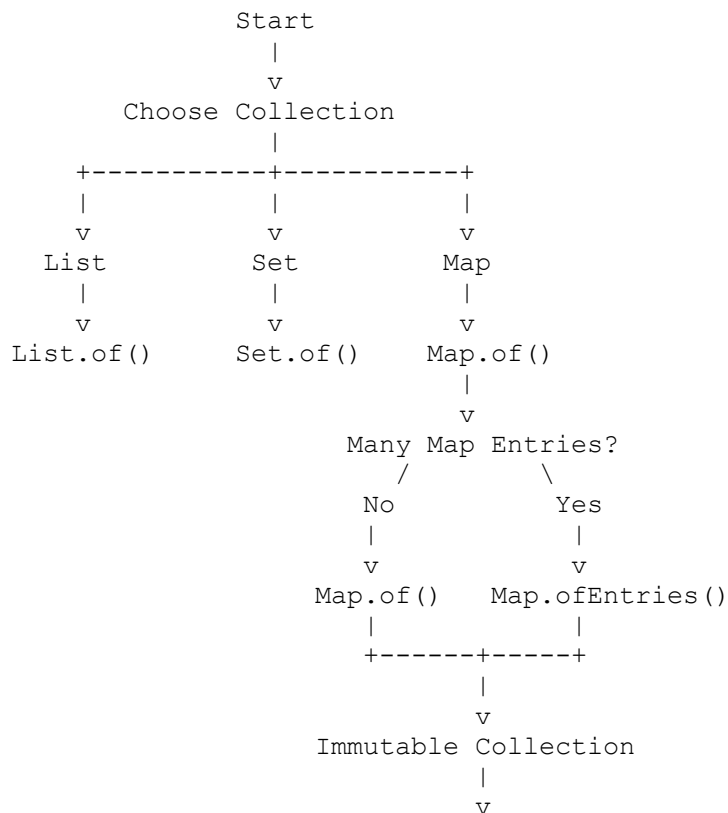
```
Map<Integer, String> map =
    Map.of(
        1, "Java",
        2, "Python",
        3, "C++"
    );
```

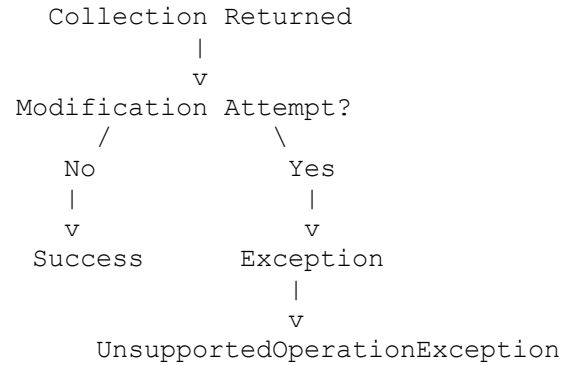
Map with Map.Entry

For larger maps:

```
Map<Integer, String> map =
    Map.ofEntries(
        Map.entry(1, "Java"),
        Map.entry(2, "Python"),
        Map.entry(3, "C++")
    );
```

4. Flowchart





5. Java Code Example

```
import java.util.List;
import java.util.Set;
import java.util.Map;

public class CollectionFactories {

    public static void main(String[] args) {

        // Immutable List
        List<String> languages =
            List.of("Java", "Python", "C++");

        // Immutable Set
        Set<String> technologies =
            Set.of("Java", "Spring", "Hibernate");

        // Immutable Map
        Map<Integer, String> versions =
            Map.of(
                8, "Java 8",
                9, "Java 9",
                17, "Java 17"
            );

        System.out.println("List: " + languages);
        System.out.println("Set: " + technologies);
        System.out.println("Map: " + versions);
    }
}
```

Output

```
List: [Java, Python, C++]
Set: [Java, Spring, Hibernate]
Map: {8=Java 8, 9=Java 9, 17=Java 17}
```

6. Demonstrating Immutability

The important characteristic of these factory-created collections is that they cannot be structurally modified.

```
import java.util.List;

public class CollectionFactories {

    public static void main(String[] args) {

        List<String> languages =
            List.of("Java", "Python", "C++");

        System.out.println(languages);

        languages.add("Go");
    }
}
```

The modification attempt:

```
languages.add("Go");
```

causes:

```
java.lang.UnsupportedOperationException
```

The flow is:

```
List.of(...)
|
v
Immutable List
|
v
languages.add("Go")
|
v
Modification rejected
|
v
UnsupportedOperationException
```

7. Set.of() Example

```
import java.util.Set;

public class CollectionFactories {

    public static void main(String[] args) {

        Set<String> languages =
            Set.of("Java", "Python", "C++");

        System.out.println(languages);
    }
}
```

Output:

[Java, Python, C++]

A Set cannot contain duplicate elements, so this is invalid:

```
Set<String> languages =
    Set.of("Java", "Python", "Java");
```

It results in:

java.lang.IllegalArgumentException

because "Java" occurs twice.

8. Map.of() Example

```
import java.util.Map;

public class CollectionFactories {

    public static void main(String[] args) {

        Map<Integer, String> languages =
            Map.of(
                1, "Java",
                2, "Python",
                3, "C++"
            );

        System.out.println(languages);
    }
}
```

Output:

```
{1=Java, 2=Python, 3=C++}
```

Keys cannot be duplicated:

```
Map<Integer, String> languages =  
    Map.of(  
        1, "Java",  
        1, "Python"  
    );
```

This results in:

```
java.lang.IllegalArgumentException
```

9. Map.ofEntries() Example

Map.of() is convenient for a small number of entries. For more entries, Map.ofEntries() can be easier to read.

```
import java.util.Map;  
  
public class CollectionFactories {  
    public static void main(String[] args) {  
        Map<Integer, String> languages =  
            Map.ofEntries(  
                Map.entry(1, "Java"),  
                Map.entry(2, "Python"),  
                Map.entry(3, "C++"),  
                Map.entry(4, "JavaScript")  
            );  
        System.out.println(languages);  
    }  
}
```

Output:

```
{1=Java, 2=Python, 3=C++, 4=JavaScript}
```

The relationship is:

```
Map.entry()  
|  
v
```

key + value
|
v
Map.ofEntries()
|
v
Immutable Map

10. Important Characteristics

1. Immutable

```
List<String> list = List.of("A", "B");
```

Operations such as:

```
list.add("C");  
list.remove("A");  
list.clear();
```

are not permitted.

2. null is not allowed

This is invalid:

```
List<String> list =  
    List.of("Java", null);
```

Factory-created collections reject null elements.

Similarly:

```
Map<String, Integer> map =  
    Map.of("Java", null);
```

is invalid.

3. Set.of() does not allow duplicate elements

```
Set.of("A", "B", "A");
```

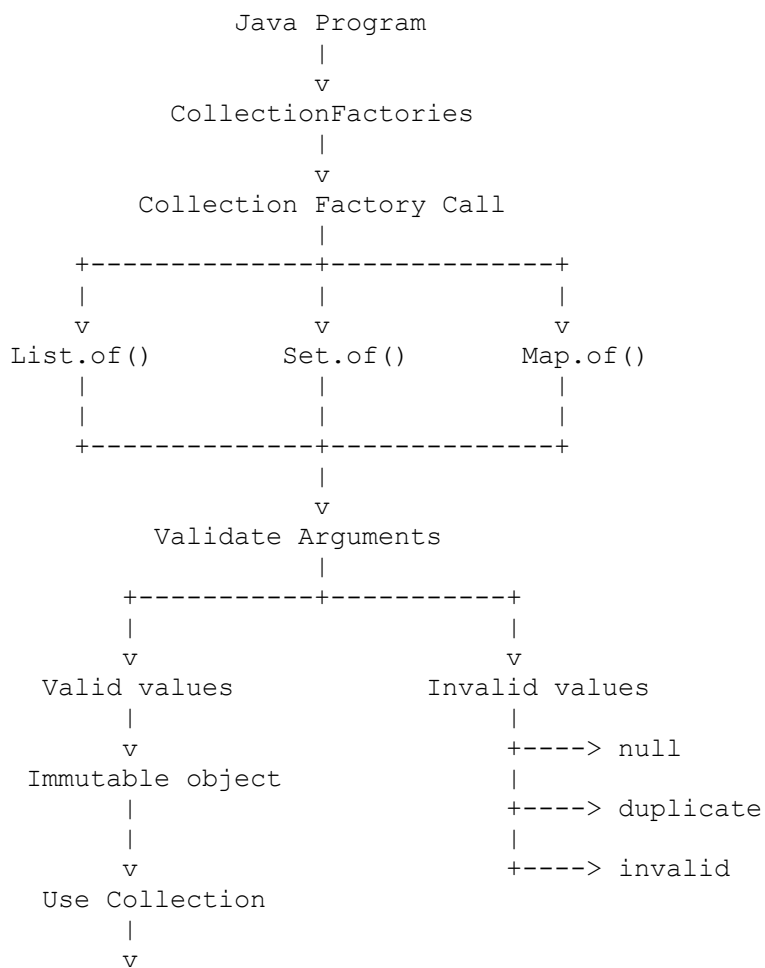
causes an `IllegalArgumentException`.

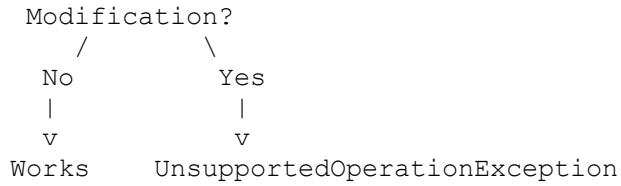
4. Map.of() does not allow duplicate keys

```
Map.of(  
    "A", 1,  
    "A", 2  
);
```

causes an `IllegalArgumentException`.

11. Complete Execution Flow



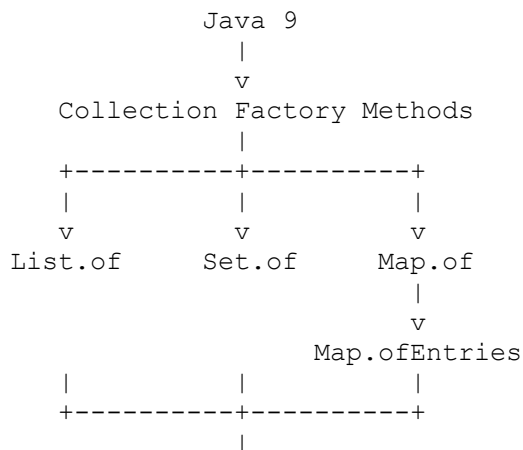


12. Before Java 9 vs Java 9

Requirement	Before Java 9	Java 9
Create list	Arrays.asList(...)	List.of(...)
Unmodifiable list	Collections.unmodifiableList(...)	List.of(...)
Create set	More boilerplate	Set.of(...)
Create map	More verbose	Map.of(...)
Create multi-entry map	Verbose	Map.ofEntries(...)
null elements	Depends on collection	Not allowed by factory methods
Structural modification	Depends on implementation	Not supported

13. Key Concept

The central idea can be remembered as:



In one sentence

Java 9 collection factory methods (`List.of()`, `Set.of()`, `Map.of()`, and `Map.ofEntries()`) provide a concise way to create immutable collections without the boilerplate traditionally required to make collections unmodifiable.

Java 9 — Stream `takeWhile()` / `dropWhile()`

Java Version: 9

Year: 2017

Feature: Stream `takeWhile()` / `dropWhile()`

UniqueClassName: `StreamTakeDropWhile`

Java 9 added two useful methods to the Stream API:

`takeWhile()`
`dropWhile()`

They are **condition-based operations** that work particularly well with **ordered streams**.

1. `takeWhile()` Definition

`takeWhile()` takes elements from the beginning of a stream **as long as the supplied condition remains true**.

As soon as the predicate becomes false, `takeWhile()` stops taking elements.

Syntax

```
stream.takeWhile(predicate)
```

Example

```
List<Integer> numbers =  
    List.of(2, 4, 6, 8, 11, 12, 14);
```

```
numbers.stream()
    .takeWhile(n -> n % 2 == 0)
    .forEach(System.out::println);
```

Output:

```
2
4
6
8
```

Why?

```
2  → even → TAKE
4  → even → TAKE
6  → even → TAKE
8  → even → TAKE
11 → odd  → STOP
12 → not examined by takeWhile
14 → not examined by takeWhile
```

The important point is that `takeWhile()` operates on the **initial contiguous portion** of the stream that satisfies the predicate.

2. `dropWhile()` Definition

`dropWhile()` does the opposite.

It skips elements from the beginning of a stream **as long as the predicate remains true**.

Once the predicate becomes false, it stops dropping and returns that element **and the remaining elements**.

Syntax

```
stream.dropWhile(predicate)
```

Example

```
List<Integer> numbers =
    List.of(2, 4, 6, 8, 11, 12, 14);

numbers.stream()
    .dropWhile(n -> n % 2 == 0)
    .forEach(System.out::println);
```

Output:

11
12
14

Flow:

2 → even → DROP
4 → even → DROP
6 → even → DROP
8 → even → DROP
11 → odd → STOP DROPPING
12 → KEEP
14 → KEEP

3. takeWhile() vs dropWhile()

Feature	takeWhile()	dropWhile()
Purpose	Takes elements while condition is true	Drops elements while condition is true
Starts from	Beginning	Beginning
When predicate becomes false	Stops taking	Stops dropping
Remaining elements	Excluded	Included
Returns	Stream	Stream
Introduced	Java 9	Java 9

Think of them as:

takeWhile()

↓

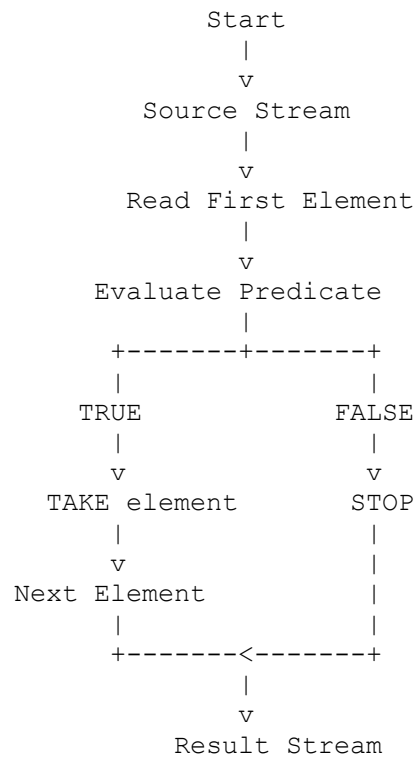
"Give me elements UNTIL the condition becomes false."

dropWhile()

↓

"Skip elements UNTIL the condition becomes false."

4. Flowchart — takeWhile()



Example flow

Input:

[2, 4, 6, 8, 11, 12, 14]

Predicate:

$$n \rightarrow n \% 2 == 0$$

2 $\rightarrow$ TRUE $\rightarrow$ TAKE

4 → TRUE → TAKE

6 $\rightarrow$ TRUE $\rightarrow$ TAKE

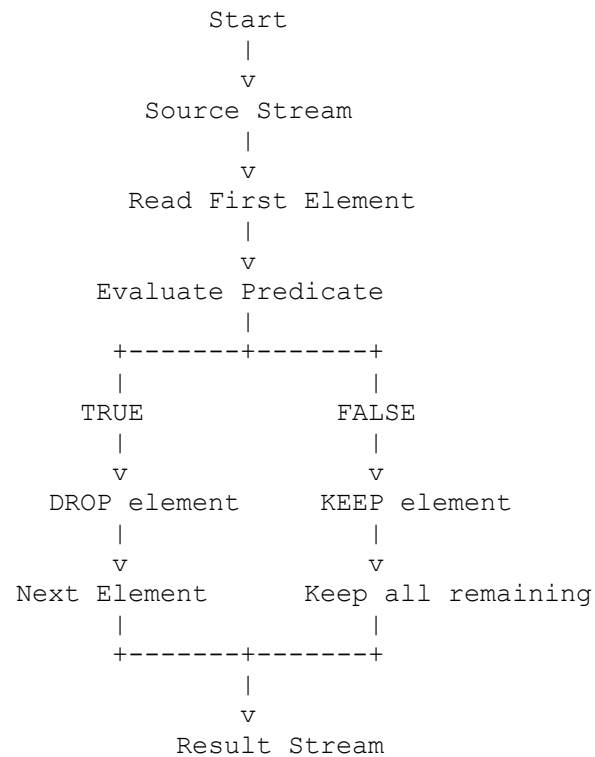
8 → TRUE → TAKE

11 $\rightarrow \text{FALSE} \rightarrow \text{STOP}$

Result:

[2, 4, 6, 8]

5. Flowchart — dropWhile()



Example flow

Input:

[2, 4, 6, 8, 11, 12, 14]

Predicate:

`n -> n % 2 == 0`

2	→	TRUE	→	DROP
4	→	TRUE	→	DROP
6	→	TRUE	→	DROP
8	→	TRUE	→	DROP
11	→	FALSE	→	KEEP
12	→	KEEP		
14	→	KEEP		

Result:

[11, 12, 14]

6. Complete Java Code Example

```
import java.util.List;

public class StreamTakeDropWhile {

    public static void main(String[] args) {

        List<Integer> numbers =
            List.of(2, 4, 6, 8, 11, 12, 14);

        System.out.println("Original List:");
        System.out.println(numbers);

        // takeWhile()
        System.out.println("\ntakeWhile() result:");

        numbers.stream()
            .takeWhile(n -> n % 2 == 0)
            .forEach(System.out::println);

        // dropWhile()
        System.out.println("\ndropWhile() result:");

        numbers.stream()
            .dropWhile(n -> n % 2 == 0)
            .forEach(System.out::println);
    }
}
```

Output

```
Original List:
[2, 4, 6, 8, 11, 12, 14]
```

```
takeWhile() result:
2
4
6
8
```

```
dropWhile() result:
11
12
14
```

7. Understanding the Algorithm

For:

```
numbers.stream()  
    .takeWhile(n -> n < 10)
```

the conceptual algorithm is:

1. Create stream
2. Read first element
3. Test $n < 10$
4. If TRUE → send element to result
5. Read next element
6. Continue testing
7. If FALSE → terminate takeWhile operation
8. Return resulting stream

For:

```
numbers.stream()  
    .dropWhile(n -> n < 10)
```

the algorithm is:

1. Create stream
 2. Read first element
 3. Test $n < 10$
 4. If TRUE → discard element
 5. Read next element
 6. Continue testing
 7. When FALSE occurs → stop dropping
 8. Keep that element
 9. Keep all subsequent elements
 10. Return resulting stream
-

8. Important Difference from filter()

This is one of the most important concepts.

Suppose:

```
List<Integer> numbers =  
    List.of(2, 4, 6, 11, 8, 10);
```

takeWhile()

```
numbers.stream()
```

```
.takeWhile(n -> n % 2 == 0)
.forEach(System.out::println);
```

Output:

```
2
4
6
```

It stops at 11.

filter()

```
numbers.stream()
    .filter(n -> n % 2 == 0)
    .forEach(System.out::println);
```

Output:

```
2
4
6
8
10
```

Why?

`filter()` examines elements independently:

```
2  → TRUE  → KEEP
4  → TRUE  → KEEP
6  → TRUE  → KEEP
11 → FALSE → REMOVE
8  → TRUE  → KEEP
10 → TRUE  → KEEP
```

Whereas `takeWhile()` is based on the **initial sequence**:

```
2  → TRUE  → TAKE
4  → TRUE  → TAKE
6  → TRUE  → TAKE
11 → FALSE → STOP
8  → NOT REACHED
10 → NOT REACHED
```

Therefore:

```
filter()
    → Can continue after FALSE.
```

```
takeWhile()
```

→ Stops at the first FALSE for an ordered stream.

9. takeWhile() and dropWhile() Together

They can be used together.

```
List<Integer> numbers =  
    List.of(2, 4, 6, 8, 11, 12, 14);
```

Example:

```
numbers.stream()  
    .dropWhile(n -> n < 10)  
    .takeWhile(n -> n < 15)  
    .forEach(System.out::println);
```

Flow:

```
Original:  
[2, 4, 6, 8, 11, 12, 14]
```

```
dropWhile(n < 10)  
    ↓  
[11, 12, 14]
```

```
takeWhile(n < 15)  
    ↓  
[11, 12, 14]
```

10. Ordered vs Unordered Streams

The behavior is especially intuitive for an **ordered stream**, where elements have a defined encounter order.

For example:

```
List<Integer> numbers =  
    List.of(2, 4, 6, 8, 11, 12);
```

has a defined order.

Conceptually:

2 → 4 → 6 → 8 → 11 → 12

`takeWhile()` and `dropWhile()` operate relative to this encounter order.

With unordered streams, the behavior does not have the same straightforward "first elements" interpretation, and the resulting set of elements may be nondeterministic.

11. `takeWhile()` vs `filter()` vs `dropWhile()`

Operation	2,4,6,8,11,12,14 with $n \% 2 == 0$
<code>takeWhile()</code>	[2,4,6,8]
<code>dropWhile()</code>	[11,12,14]
<code>filter()</code>	[2,4,6,8,12,14]

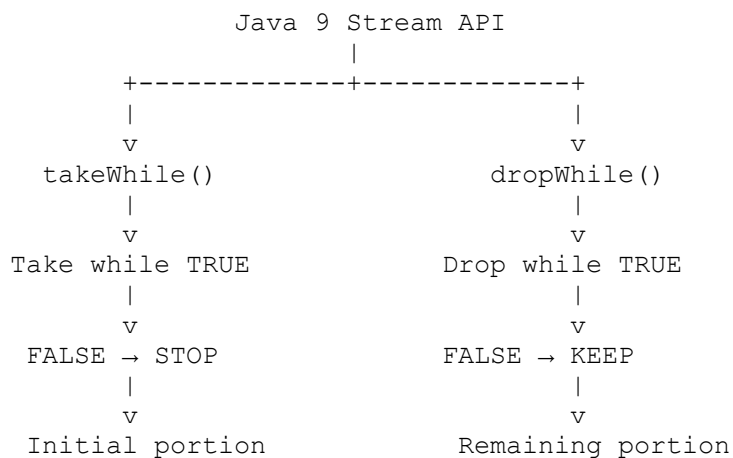
The distinction is:

```
takeWhile()
    TAKE from beginning until FALSE

dropWhile()
    DROP from beginning until FALSE

filter()
    CHECK EVERY element independently
```

12. Key Concept



In one sentence

Java 9's takeWhile() takes elements from the beginning of an ordered stream while a predicate is true, whereas dropWhile() skips elements from the beginning while the predicate is true and then retains the remaining elements.

Java 9 — Private Interface Methods

Java Version: 9

Year: 2017

Feature: Private Interface Methods

UniqueClassName: PrivateInterfaceMethods

1. Definition

Private interface methods were introduced in **Java 9**.

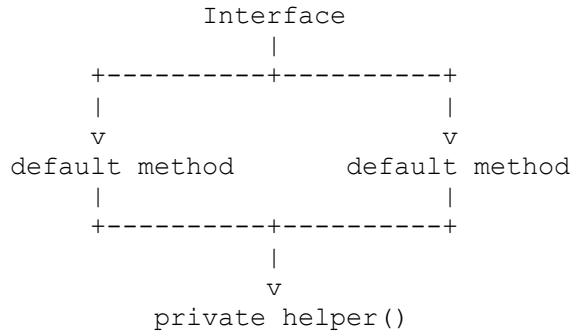
They allow an interface to contain private methods that can be used **only inside that same interface**.

Their primary purpose is to allow **code reuse between default and static methods** without exposing helper methods as part of the interface's public API.

Basic syntax

```
interface MyInterface {  
  
    default void methodA() {  
        helper();  
    }  
  
    default void methodB() {  
        helper();  
    }  
  
    private void helper() {  
        System.out.println("Common logic");  
    }  
}
```

The important relationship is:



The implementing class **cannot directly call** the private method.

2. Why Private Interface Methods Were Introduced

Java 8 introduced:

default
static

methods inside interfaces.

For example:

```
interface Calculator {  
    default void add() {  
        // logic  
    }  
    default void subtract() {  
        // logic  
    }  
}
```

Suppose both methods need the same internal logic.

Before Java 9, an interface had no private instance methods for sharing that implementation.

Java 9 solves this:

```
interface Calculator {  
    default void add() {
```

```

    calculate();
}

default void subtract() {
    calculate();
}

private void calculate() {
    System.out.println("Common calculation logic");
}
}

```

Therefore, private interface methods primarily provide **implementation-level code reuse**.

3. Characteristics

Characteristic	Private Interface Method
Introduced	Java 9
Access modifier	Private
Accessible from	Same interface only
Accessible from implementing class	No
Inherited by implementation class	No
Can be called by default methods	Yes
Can be called by interface static methods	Yes, with appropriate static declaration
Purpose	Internal code reuse
Part of public API	No

4. Types of Private Interface Methods

There are two important forms.

1. Private instance method

```

private void helper() {
    // logic
}

```

```
}
```

It can be called by default methods.

2. Private static method

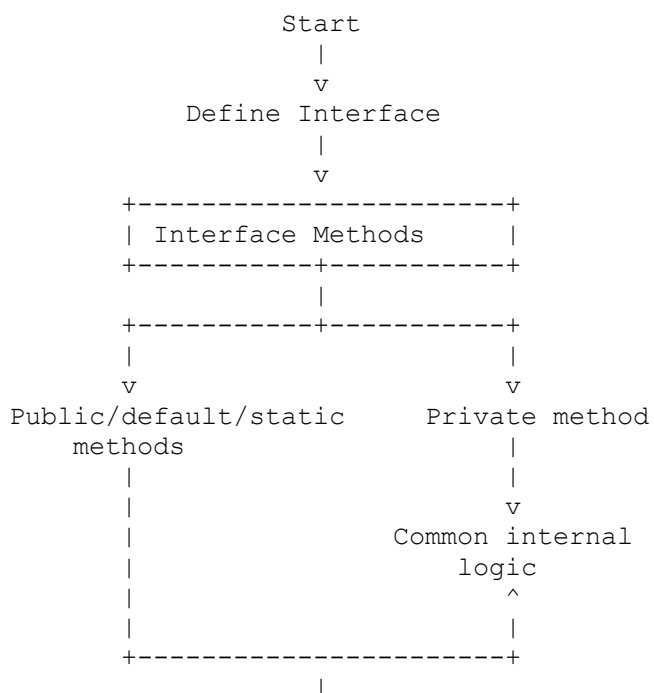
```
private static void helper() {  
    // logic  
}
```

It can be called by static methods and other appropriate interface code.

Example:

```
interface Utility {  
  
    static void methodA() {  
        helper();  
    }  
  
    private static void helper() {  
        System.out.println("Private static helper");  
    }  
}
```

5. Flowchart



```

      v
Implementing Class
      |
      v
Can access public/default
interface functionality
      |
      v
Cannot directly access
private method
      |
      v
End

```

6. Java Code Example

Let's create an interface containing two default methods that reuse a private method.

```

interface Calculator {

    default void add(int a, int b) {
        validate(a, b);
        System.out.println("Addition = " + (a + b));
    }

    default void multiply(int a, int b) {
        validate(a, b);
        System.out.println("Multiplication = " + (a * b));
    }

    private void validate(int a, int b) {
        System.out.println(
            "Validating values: " + a + ", " + b
        );
    }
}

public class PrivateInterfaceMethods
    implements Calculator {

    public static void main(String[] args) {

        PrivateInterfaceMethods calculator =
            new PrivateInterfaceMethods();

        calculator.add(10, 20);
        calculator.multiply(10, 20);
    }
}

```

Output

Validating values: 10, 20
Addition = 30
Validating values: 10, 20
Multiplication = 200

7. Execution Flow of the Example

When:

`calculator.add(10, 20);`

is executed:

```
calculator.add(10, 20)
  |
  v
Calculator.add()
  |
  v
  validate(10, 20)
    |
    v
private validate()
  |
  v
Validation logic
  |
  v
Addition logic
  |
  v
Output: 30
```

When:

`calculator.multiply(10, 20);`

is executed:

```
calculator.multiply(10, 20)
  |
  v
Calculator.multiply()
  |
  v
  validate(10, 20)
```

```

      |
      v
private validate()
      |
      v
  Validation logic
      |
      v
Multiplication logic
      |
      v
    Output: 200

```

The same private method is reused by both default methods.

8. Private Static Interface Method

Java 9 also allows private **static** methods in interfaces.

```

interface MessageService {

    static void sendMessage() {
        String message = createMessage();

        System.out.println(message);
    }

    private static String createMessage() {
        return "Hello from private static interface method!";
    }
}

public class PrivateInterfaceMethods {

    public static void main(String[] args) {

        MessageService.sendMessage();
    }
}

```

Output

Hello from private static interface method!

The flow is:

```

MessageService.sendMessage()
      |
      v
    createMessage()

```

```

    |
    v
private static method
    |
    v
    Return String
    |
    v
    Print message

```

9. Private Method Cannot Be Called Outside the Interface

Consider:

```

interface Calculator {

    default void calculate() {
        helper();
    }

    private void helper() {
        System.out.println("Internal helper");
    }
}

```

This is valid:

```

default void calculate() {
    helper();
}

```

But this is **not** allowed from an implementing class:

```

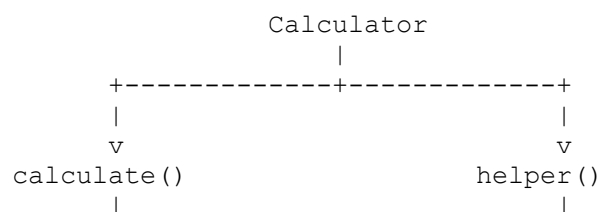
Calculator calculator = new SomeCalculator();

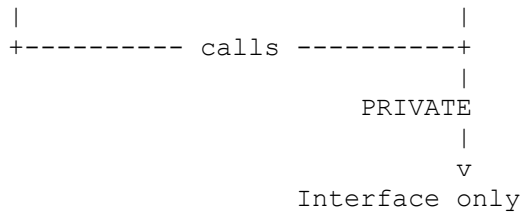
calculator.helper();

```

The private method is an implementation detail of the interface.

Conceptually:





10. Before Java 9 vs Java 9

Aspect	Before Java 9	Java 9
Interface default methods	Java 8	Yes
Interface static methods	Java 8	Yes
Private interface methods	Not supported	Supported
Code reuse between default methods	Limited	Easy
Internal helper methods	Required workarounds	private methods
Public API pollution	More likely	Reduced

11. Important Difference: default vs private

default

```
default void display() {
    System.out.println("Display");
}
```

A default method can be inherited by an implementing class.

private

```
private void helper() {
    System.out.println("Helper");
}
```

A private method is **not inherited** and cannot be directly accessed by the implementing class.

Therefore:

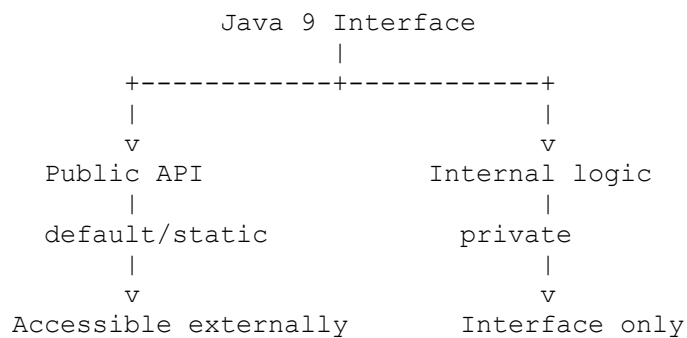
```

Interface
|
+-- default method
|   |
|   +---- Can be inherited
|
+-- private method
|   |
|   +---- Internal interface implementation only

```

12. Key Concept

The easiest way to remember private interface methods is:



In one sentence

Java 9 private interface methods allow interfaces to define reusable internal helper logic that can be called by their own default and static methods without exposing that logic to implementing classes or interface users.

Java 9 — ProcessHandle API

Java Version: 9

Year: 2017

Feature: ProcessHandle API

UniqueClassName: ProcessHandle

1. Definition

The **ProcessHandle API** was introduced in **Java 9** to provide a standard Java API for **managing and monitoring operating-system processes**.

Before Java 9, Java applications had limited and platform-dependent ways to interact with OS processes. Java 9 introduced:

`java.lang.ProcessHandle`

and:

`java.lang.ProcessHandle.Info`

With this API, Java programs can:

- Get the current process
- Get a process by its PID
- Obtain the process ID (**PID**)
- Inspect process information
- Find parent and child processes
- Check whether a process is alive
- Wait for a process to terminate
- Destroy a process
- Destroy a process forcibly
- Obtain information such as command, arguments, start time, and CPU duration

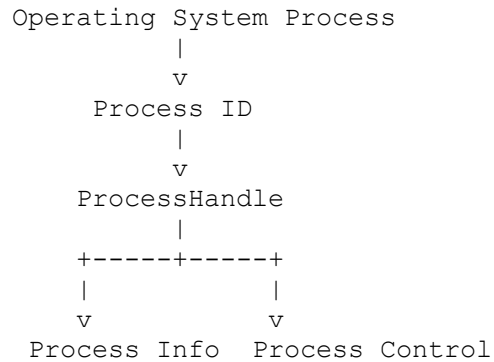
2. Basic Concept

An operating system assigns a unique **Process ID (PID)** to a running process.

For example:

```
Operating System
|
+---- Process A → PID 1001
|
+---- Process B → PID 1002
|
+---- Process C → PID 1003
```

Java 9 allows you to represent these processes using ProcessHandle.



3. ProcessHandle vs Process

Java has both Process and ProcessHandle.

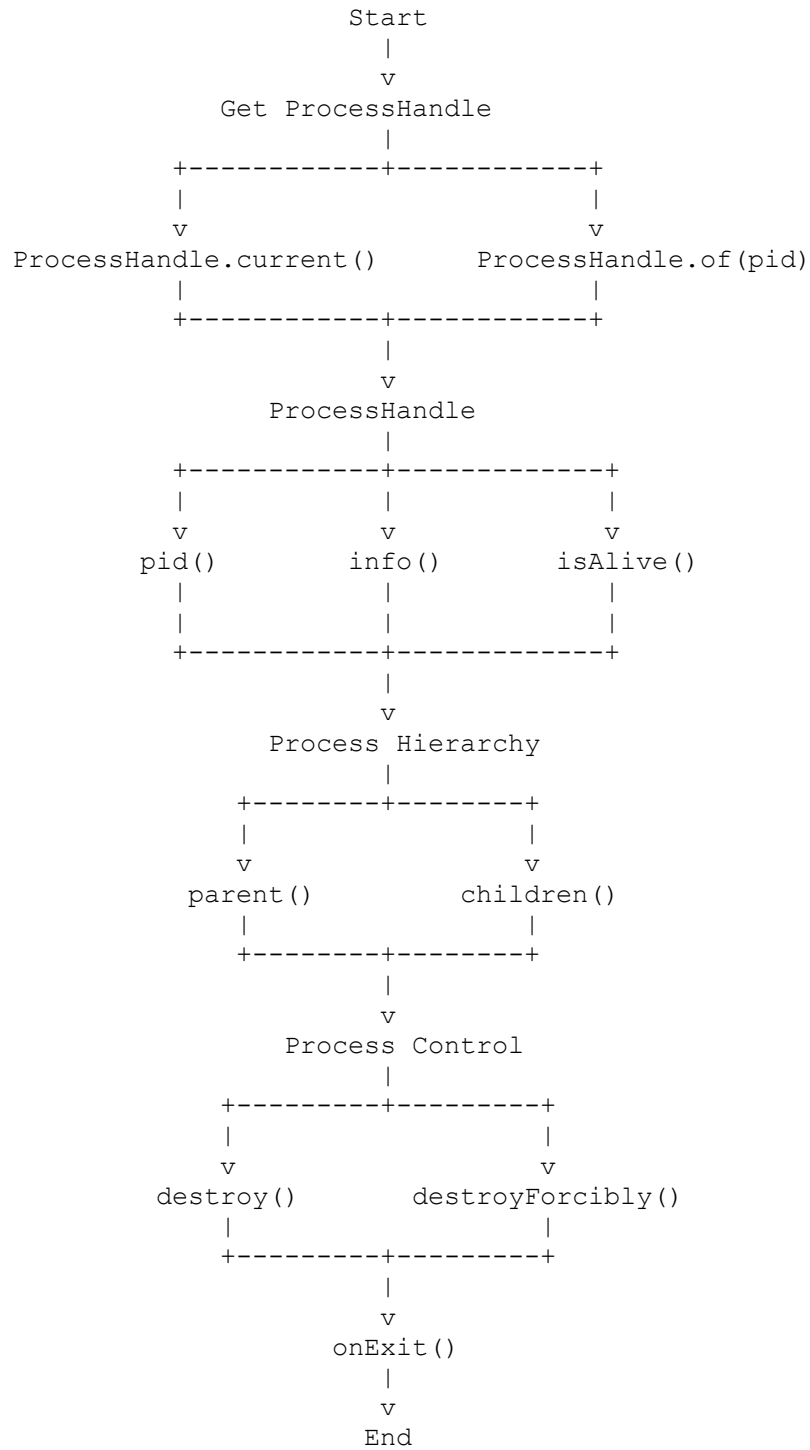
Feature	Process	ProcessHandle
Introduced	Java 1.0	Java 9
Represents OS process	Yes	Yes
Get PID	Limited/modern Java support	Yes
Parent process	Limited	Yes
Child processes	Limited	Yes
Process information	Limited	ProcessHandle.Info
Check alive	isAlive()	isAlive()
Wait for termination	waitFor()	onExit()
Destroy process	destroy()	destroy()
Force destroy	destroyForcibly()	destroyForcibly()

ProcessHandle is especially useful when you need to **inspect and manage the process hierarchy**.

4. Important Methods

Method	Purpose
ProcessHandle.current()	Returns the current process
ProcessHandle.of(pid)	Returns a ProcessHandle for a PID if available
ProcessHandle.pid()	Returns the process ID
isAlive()	Checks whether the process is running
parent()	Returns the parent process
children()	Returns direct child processes
descendants()	Returns descendant processes
info()	Returns process information
onExit()	Returns a CompletableFuture completed when the process exits
destroy()	Requests process termination
destroyForcibly()	Attempts to forcibly terminate the process
allProcesses()	Returns a stream of processes visible to the current process

5. Flowchart



6. Java Code Example — Current Process

The simplest example is obtaining the process in which the Java application is currently running.

```
public class ProcessHandle {  
  
    public static void main(String[] args) {  
  
        java.lang.ProcessHandle process =  
            java.lang.ProcessHandle.current();  
  
        System.out.println("Current PID: "  
            + process.pid());  
  
        System.out.println("Is Alive: "  
            + process.isAlive());  
    }  
}
```

Example output

```
Current PID:      15432  
Is Alive:         true
```

The PID will be different on each execution.

7. Getting Process Information

ProcessHandle.Info provides information about a process.

```
public class ProcessHandle {  
  
    public static void main(String[] args) {  
  
        java.lang.ProcessHandle process =  
            java.lang.ProcessHandle.current();  
  
        java.lang.ProcessHandle.Info info =  
            process.info();  
  
        System.out.println("PID: "  
            + process.pid());  
  
        System.out.println("Command: "  
            + info.command().orElse("Unknown"));  
  
        System.out.println("Command Line: "  
            + info.commandLine().orElse("Unknown"));  
    }  
}
```

```

        System.out.println("Arguments: "
            + info.arguments()
                .map(argsArray -> String.join(" ", argsArray))
                .orElse("None"));
    }
}

```

Example output:

```

PID:          15432
Command:      C:\Program Files\Java\jdk-...\bin\java.exe
Command Line: ...
Arguments:    None

```

Some information is returned through Optional because the operating system may not provide it.

8. Getting the Parent Process

A process normally has a parent process.

For example:

```

Operating System
|
v
Parent Process
|
v
Java Process
|
v
ProcessHandle

```

Java code:

```

public class ProcessHandle {

    public static void main(String[] args) {

        java.lang.ProcessHandle current =
            java.lang.ProcessHandle.current();

        current.parent().ifPresent(parent -> {

            System.out.println(
                "Current PID: " + current.pid());

            System.out.println(

```

```

        "Parent PID: " + parent.pid());
    });
}
}

```

Possible output:

Current PID: 15432

Parent PID: 12080

9. Finding Child Processes

You can obtain the direct children of a process.

```

public class ProcessHandle {

    public static void main(String[] args) {

        java.lang.ProcessHandle current =
            java.lang.ProcessHandle.current();

        System.out.println(
            "Current PID: " + current.pid());

        current.children().forEach(child -> {

            System.out.println(
                "Child PID: " + child.pid());

        });
    }
}

```

Flow:

Current Process

```

|
+---- Child Process 1
|
+---- Child Process 2
|
+---- Child Process 3

```

10. Finding Descendant Processes

`children()` returns **direct children**.

`descendants()` goes deeper into the process hierarchy.

```
Current Process
|
+---- Child A
|      |
|      +---- Grandchild A1
|
+---- Child B
|      |
|      +---- Grandchild B1
```

Using:

`current.descendants()`

can provide the descendant processes in that hierarchy.

Example:

```
public class ProcessHandle {

    public static void main(String[] args) {

        java.lang.ProcessHandle current =
            java.lang.ProcessHandle.current();

        System.out.println(
            "Current PID: " + current.pid());

        current.descendants().forEach(process -> {

            System.out.println(
                "Descendant PID: "
                + process.pid());

        });

    }

}
```

11. Checking Whether a Process Is Alive

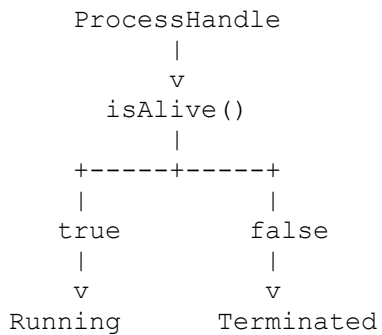
Use:

`isAlive()`

Example:

```
public class ProcessHandle {  
    public static void main(String[] args) {  
        java.lang.ProcessHandle process =  
            java.lang.ProcessHandle.current();  
  
        if (process.isAlive()) {  
            System.out.println(  
                "Process is running.");  
        } else {  
            System.out.println(  
                "Process has terminated.");  
        }  
    }  
}
```

Flow:

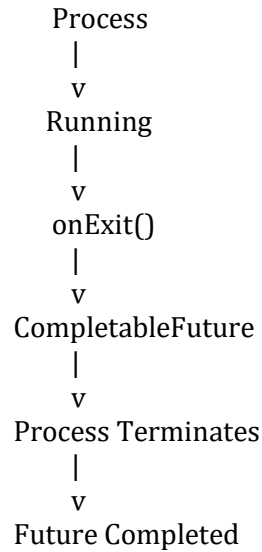


12. Waiting for Process Termination

`onExit()` returns a `CompletableFuture<ProcessHandle>`.

This allows asynchronous handling of process termination.

Conceptually:



Example:

```
import java.util.concurrent.CompletableFuture;

public class ProcessHandle {

    public static void main(String[] args) {

        java.lang.ProcessHandle process =
            java.lang.ProcessHandle.current();

        CompletableFuture<java.lang.ProcessHandle>
            future = process.onExit();

        System.out.println(
            "Waiting for process termination...");

        future.thenAccept(p ->
            System.out.println(
                "Process terminated: "
                + p.pid()));
    }
}
```

For the current Java process, the program's lifetime and the completion of the future are naturally tied together, so `onExit()` is generally more useful with a separately launched process.

13. Starting Another Process

ProcessHandle is primarily for **representing and managing processes**. A process can be launched using ProcessBuilder.

Example:

```
import java.io.IOException;

public class ProcessHandle {

    public static void main(String[] args)
        throws IOException {

        Process process = new ProcessBuilder(
            "java",
            "-version"
        ).inheritIO().start();

        java.lang.ProcessHandle handle =
            process.toHandle();

        System.out.println(
            "Started PID: " + handle.pid());

        System.out.println(
            "Is Alive: " + handle.isAlive());

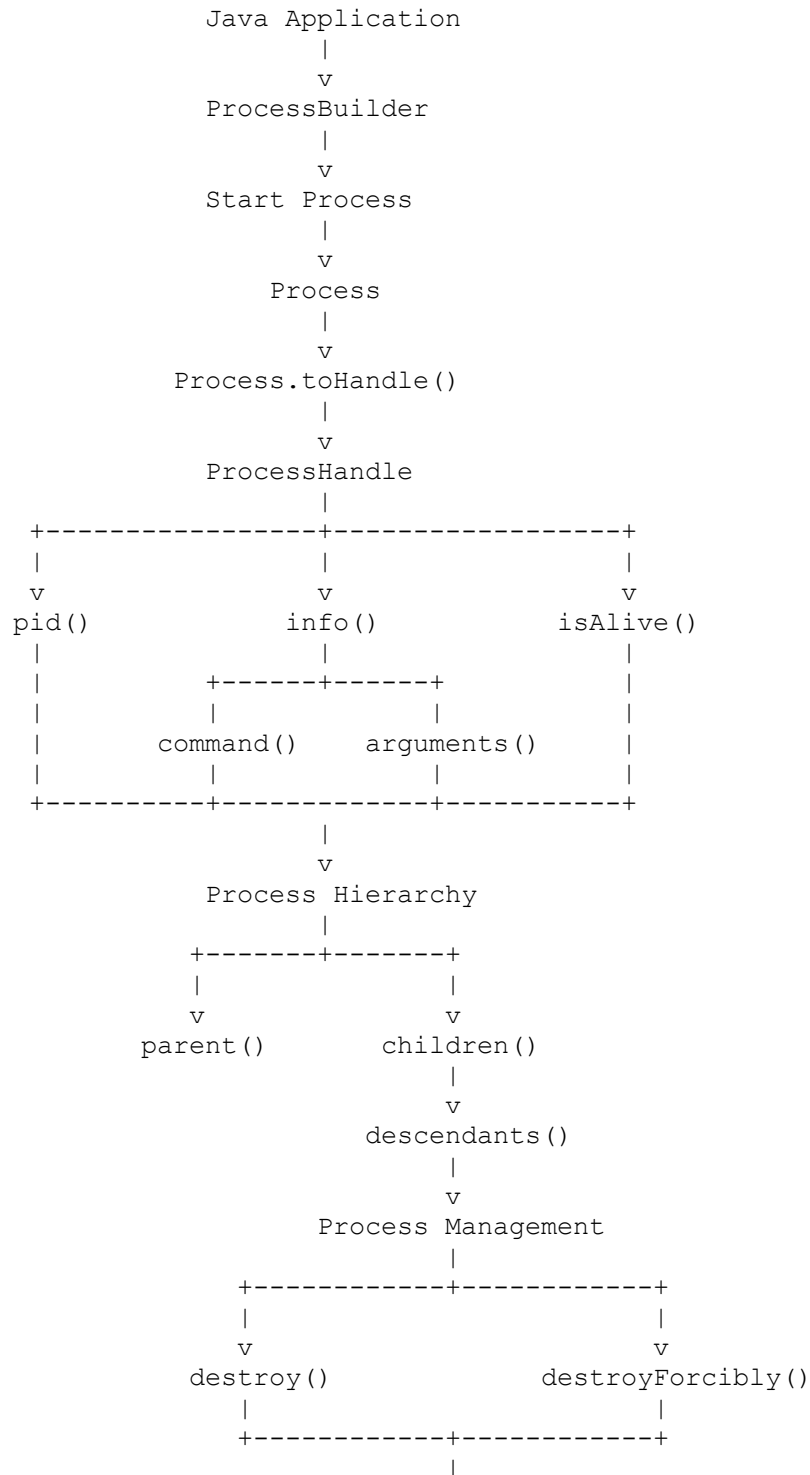
        handle.onExit().thenAccept(p ->
            System.out.println(
                "Process " + p.pid()
                + " has terminated."));
    }
}
```

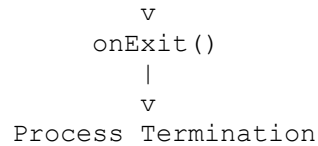
The important relationship is:

```
ProcessBuilder
    |
    v
Start OS Process
    |
    v
Process
    |
    v
Process.toHandle()
    |
    v
ProcessHandle
    |
    +---- pid()
    +---- info()
    +---- isAlive()
```

```
+---- onExit()  
+---- destroy()
```

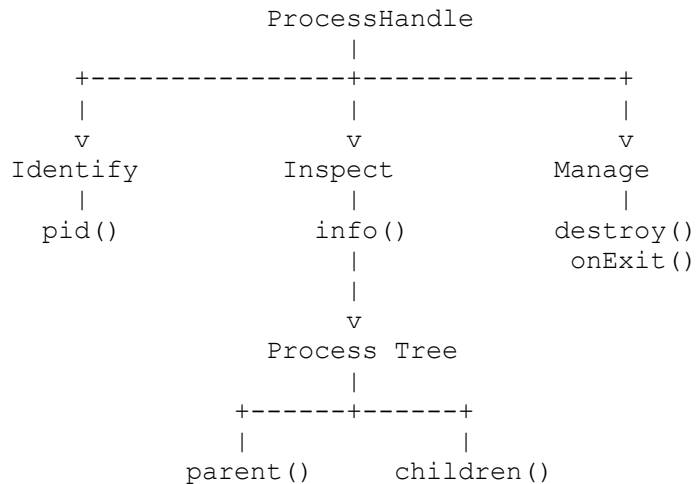
14. Complete ProcessHandle Flow





15. Key Concept

The easiest way to remember the Java 9 ProcessHandle API is:



In one sentence

Java 9's ProcessHandle API provides a standard way to identify, inspect, monitor, navigate, and manage operating-system processes and their process hierarchy from Java.

Java 9 — StackWalker API

Java Version: 9

Year: 2017

Feature: StackWalker API

UniqueClassName: StackWalker

1. Definition

The **StackWalker API** was introduced in **Java 9** to provide an efficient and controlled way to **walk through the current thread's stack frames**.

It is available in:

`java.lang.StackWalker`

A **stack frame** represents an active method call on the current thread's call stack.

For example:

```
main()
|
v
methodA()
|
v
methodB()
|
v
methodC()
```

StackWalker allows Java code to inspect this call stack programmatically.

It can be used to obtain information such as:

- Class name
- Method name
- File name
- Line number
- Declaring class
- Whether the method is native
- Caller information

2. Why StackWalker Was Introduced

Before Java 9, developers commonly used:

```
Thread.currentThread().getStackTrace()
```

or:

```
new Throwable().getStackTrace()
```

These approaches could be less convenient or less efficient when only a small portion of the stack was required.

Java 9 introduced:

StackWalker

which provides a **lazy, controlled stack traversal API**.

Instead of necessarily creating an array containing the entire stack, you can process stack frames as needed.

3. Basic Syntax

Create a StackWalker:

```
StackWalker walker = StackWalker.GetInstance();
```

Walk through stack frames:

```
walker.forEach(frame -> {  
    System.out.println(frame);  
});
```

Or:

```
walker.walk(stream ->  
    stream.forEach(System.out::println)  
);
```

4. What Is a Stack Frame?

A stack frame represents one method invocation in the current thread.

Suppose:

```
main()  
↓  
methodA()  
↓  
methodB()  
↓  
methodC()
```

The stack conceptually contains:

```
+-----+  
| methodC() frame |  
+-----+  
| methodB() frame |
```

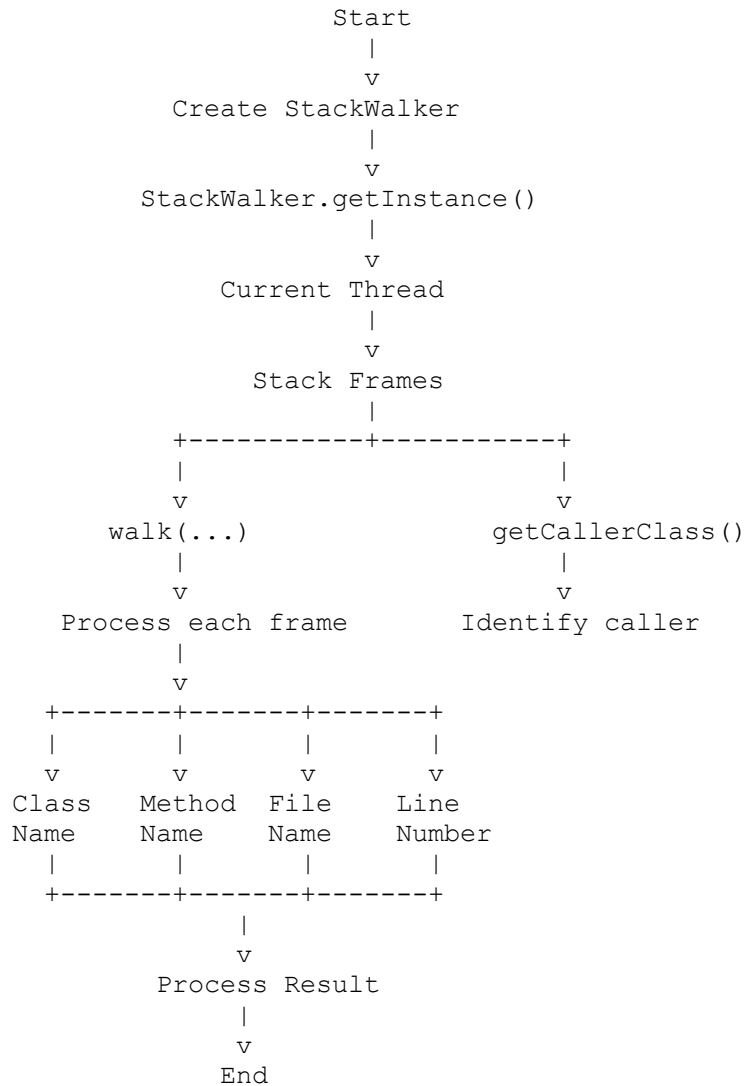
```
+-----+
| methodA() frame |
+-----+
| main() frame    |
+-----+
```

Each StackWalker.StackFrame can provide information about one of these frames.

5. Important StackFrame Methods

Method	Purpose
getClassName()	Returns the class name
getMethodName()	Returns the method name
getFileName()	Returns source file name
getLineNumber()	Returns source line number
getDeclaringClass()	Returns the declaring class
isNativeMethod()	Indicates whether the method is native
toStackTraceElement()	Converts the frame to StackTraceElement

6. Flowchart



7. Basic Java Code Example

```
public class StackWalker {

    public static void main(String[] args) {

        StackWalker walker =
            java.lang.StackWalker.getInstance();

        methodA(walker);
    }

    static void methodA(
        java.lang.StackWalker walker) {
```

```

        methodB(walker);
    }

    static void methodB(
        java.lang.StackWalker walker) {

        walker.forEach(frame -> {

            System.out.println(
                frame.getClassName()
                + " -> "
                + frame.getMethodName());
        });
    }
}

```

Possible output:

```

StackWalker -> methodB
StackWalker -> methodA
StackWalker -> main

```

The exact output can vary depending on the execution environment.

8. Execution Flow

The program calls methods in this sequence:

```

main()
|
v
methodA()
|
v
methodB()
|
v
StackWalker.forEach()

```

At methodB(), the current stack can be viewed as:

```

+-----+
| StackWalker.forEach |
+-----+
| StackWalker.methodB |
+-----+
| StackWalker.methodA |
+-----+

```

```
| StackWalker.main      |  
+-----+
```

StackWalker traverses the available frames from the current execution point toward the caller.

9. Getting Detailed Frame Information

You can obtain individual pieces of information:

```
public class StackWalker {  
    public static void main(String[] args) {  
        java.lang.StackWalker walker =  
            java.lang.StackWalker.getInstance();  
        walker.forEach(frame -> {  
            System.out.println(  
                "Class  : "  
                + frame.getClassName());  
            System.out.println(  
                "Method : "  
                + frame.getMethodName());  
            System.out.println(  
                "File   : "  
                + frame.getFileName());  
            System.out.println(  
                "Line   : "  
                + frame.getLineNumber());  
            System.out.println();  
        });  
    }  
}
```

Possible output:

```
Class   : StackWalker  
Method  : main  
File    : StackWalker.java  
Line    : 8
```

10. Using walk()

walk() gives you a Stream<StackFrame> for controlled processing.

Example:

```
public class StackWalker {  
    public static void main(String[] args) {  
        java.lang.StackWalker walker =  
            java.lang.StackWalker.getInstance();  
  
        walker.walk(stream ->  
            stream.limit(3)  
                .forEach(frame ->  
                    System.out.println(  
                        frame.getMethodName()  
                    )  
                )  
            );  
    }  
}
```

Here:

stream.limit(3)

means only the first three stack frames are processed.

Conceptually:

```
Complete Stack  
  |  
  v  
StackWalker  
  |  
  v  
StackFrame Stream  
  |  
  v  
limit(3)  
  |  
  v  
First 3 Frames  
  |  
  v  
forEach()
```

This is one of the advantages of the StackWalker API: **you can process only the frames you need.**

11. Getting the Immediate Caller

StackWalker can also be used to identify the caller of a method.

```
public class StackWalker {  
    private static final java.lang.StackWalker WALKER =  
        java.lang.StackWalker.getInstance(  
            java.lang.StackWalker.Option.RETAIN_CLASS_REFERENCE  
        );  
  
    public static void main(String[] args) {  
        displayCaller();  
    }  
  
    static void displayCaller() {  
        Class<?> caller =  
            WALKER.getCallerClass();  
  
        System.out.println(  
            "Caller: " + caller.getName());  
    }  
}
```

Possible output:

Caller: StackWalker

getCallerClass() is useful when a utility method needs to know **which class invoked it**.

12. StackWalker.Option

StackWalker provides options that control what information is retained.

One important option is:

RETAIN_CLASS_REFERENCE

Example:

```
java.lang.StackWalker walker =  
    java.lang.StackWalker.getInstance(  
        java.lang.StackWalker.Option.RETAIN_CLASS_REFERENCE  
    );
```

This allows operations such as:

```
walker.getClass();
```

Without retaining class references, certain class-reference operations are not available.

13. SHOW_REFLECT_FRAMES

Another option is:

SHOW_REFLECT_FRAMES

It allows reflective invocation frames to be included when walking the stack.

Example:

```
java.lang.StackWalker walker =  
    java.lang.StackWalker.getInstance(  
        java.lang.StackWalker.Option.SHOW_REFLECT_FRAMES  
    );
```

14. SHOW_HIDDEN_FRAMES

Java 9 also provides:

SHOW_HIDDEN_FRAMES

This allows hidden frames to be included in stack walking.

For example:

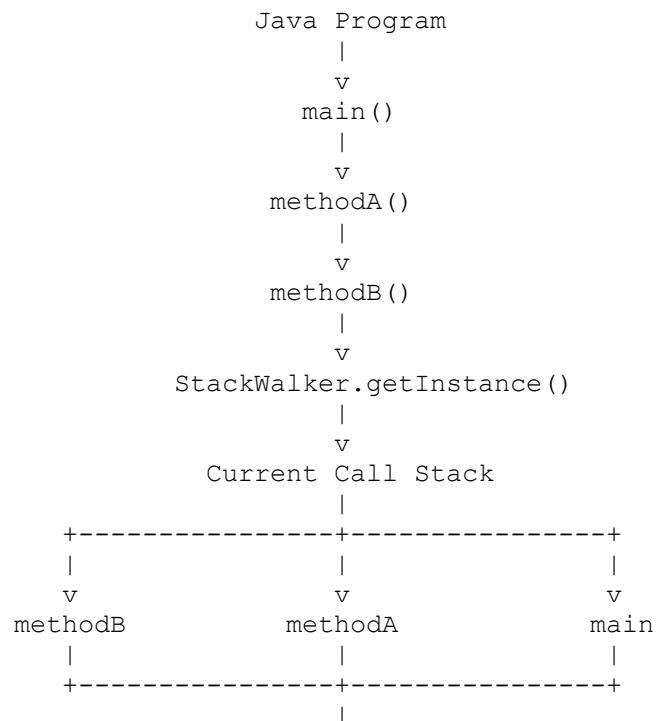
```
java.lang.StackWalker walker =  
    java.lang.StackWalker.getInstance(  
        java.lang.StackWalker.Option.SHOW_HIDDEN_FRAMES  
    );
```

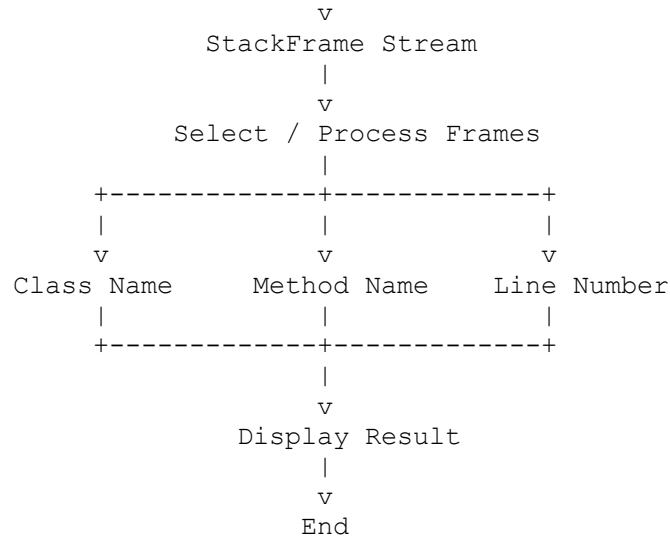
These options give developers more control over what the stack walker sees.

15. StackWalker vs getStackTrace()

Feature	Thread.getStackTrace()	StackWalker
Available since	Java 1.2	Java 9
Stack inspection	Yes	Yes
Lazy traversal	Limited	Yes
Stream-based processing	No	Yes
Limit frames easily	Less convenient	Yes
Caller class support	Less direct	getCallerClass()
Stack frame abstraction	StackTraceElement	StackFrame
Selective traversal	Less convenient	Yes
Designed for controlled stack walking	No	Yes

16. Complete Execution Flow





17. Practical Use Cases

StackWalker is particularly useful for:

```

StackWalker
|
+-- Logging
|
+-- Debugging
|
+-- Caller identification
|
+-- Framework development
|
+-- Diagnostic tools
|
+-- Tracing method calls
|
+-- Security / access checks
|
+-- Stack-aware utilities
  
```

For example, a logging utility can identify the method that invoked it:

```

Application
|
v
Service.method()
|
v
Logger.log()
|
v
StackWalker
  
```

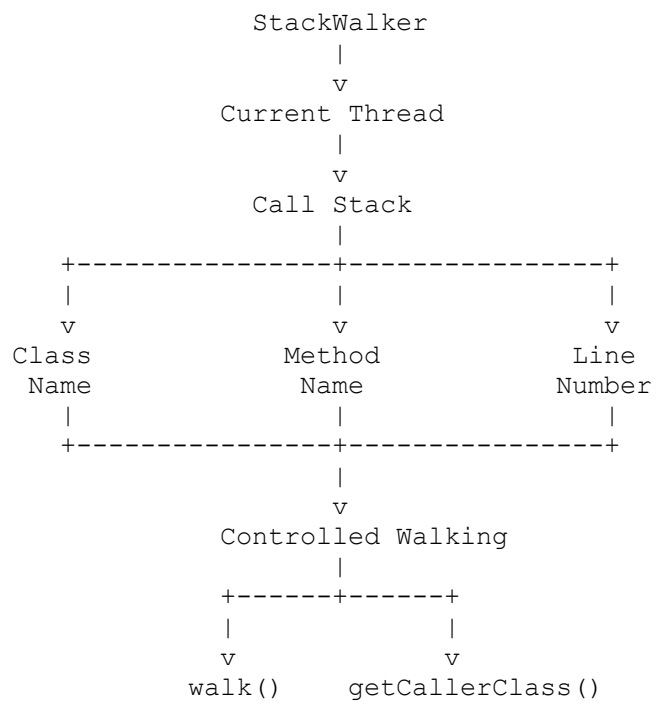
```

    |
    v
Find caller
    |
    v
Service.method()

```

18. Key Concept

The easiest way to remember StackWalker is:



In one sentence

Java 9's StackWalker API provides a controlled, efficient, and stream-based way to traverse the current thread's stack frames and inspect information about active method calls.